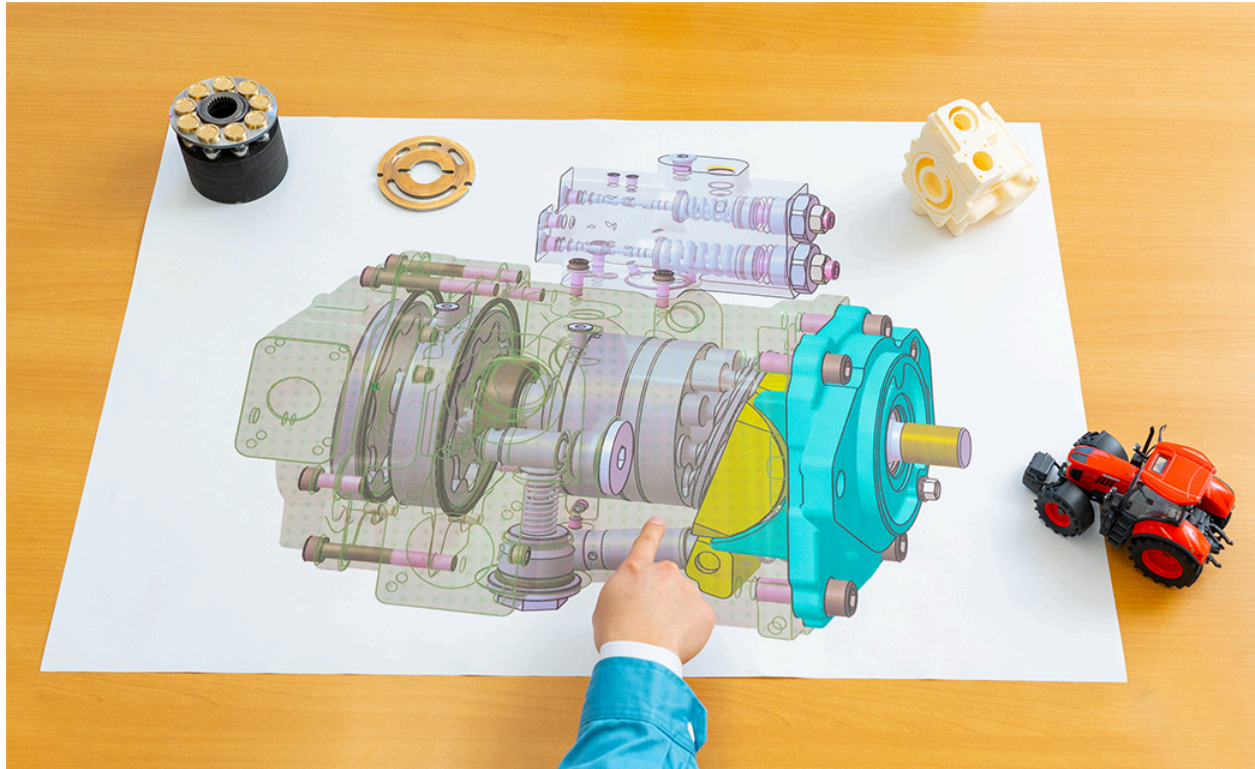




SOFTWARE REQUIREMENTS SPECIFICATION

Audrolics

Automated Calculation

Version 1.0

Prepared by: Oceans and Arrays

Members:

De Galicia, MJ

Gabot, Kelly

Jondiz, Cesar III

Migrino, Areisha Julliana

Tablizo, Paul Edward

Table of Contents

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Scope.....	3
1.3 Disclaimer.....	3
1.4 Intended Users.....	4
1.5 Definition of Terms.....	4
2. Overall Description.....	5
2.1 Product Perspective.....	5
2.2 Tech Stack.....	5
2.3 External Interface Requirements.....	5
2.4 Data Model.....	6
3. Feature 1: Schematic Diagram Maker.....	8
3.1 Element Library ("weights").....	8
3.2 UI/UX.....	9
3.3. Input Validation Rules.....	11
3.4 Curve Data Validation.....	11
4. Feature 2: Autocalculate / Anomaly Detection.....	12
4.1 Calculation Method.....	12
4.2 Workflow.....	14
4.3 Deviation Threshold.....	15
4.4 Network Validation.....	16
4.5 Anomaly Localization Algorithm.....	17
5. Feature 3: Landing Page.....	19
6. Feature 4: Login & Signup.....	19

7. Non-Functional Requirements.....	20
8. Assumptions & Dependencies.....	22
9. Recommendations.....	22
10. Functional Requirements.....	23
11. API Specification.....	24
12. Error Handling.....	27
13. Testing Requirements.....	30
13.1 Simulation Accuracy Tests.....	30
13.2 Anomaly Detection Tests.....	31
13.3 UI/UX Tests.....	31
13.4 Security Tests.....	31
14. References.....	32
Appendix A: Units & Convention.....	32
Appendix B: Hazen-Williams Worked Example.....	33
Appendix C: Anomaly Localization Pseudocode.....	34

1. Introduction

1.1 Purpose

Audrolics is a web-based system that lets field engineers build a schematic (line-and-symbol, not photorealistic) representation of a pressurized water pipe network, then uses hydraulic simulation to detect and localize anomalies such as leaks, blockages, and faulty readings. This is done by comparing simulated expected behavior against manually entered field measurements.

1.2 Scope

In scope:

- Schematic diagram builder,
- Hazen-Williams-based hydraulic simulation engine,
- Manual measurement input at node and link elements,
- Physics-based anomaly detection and pipe-segment localization,
- Landing page with product description,
- Email/password authentication.

Out of scope (documented as future work):

- ML-based condition monitoring (pattern recognition across historical data),
- Auto-measuring hardware/SCADA (Supervisory Control and Data Acquisition) sensor integration,
- AutoCAD or EPANET .inp file import,
- Real-time multi-user collaboration,
- Extended Period Simulation (EPS) / time-varying demands
- Oil-based hydraulic systems

1.3 Disclaimer

This system is being built by a non-hydraulics-expert team for hydraulics practitioners (Maynilad field engineers/technicians). All calculation methods below follow the

industry-standard approach used by EPANET, the most widely adopted water distribution modeling software, and should be reviewed by a hydraulics SME before production use.

1.4 Intended Users

Maynilad field engineers and technicians who need to:

1. Diagram a pipe segment using standard hydraulic symbols
2. Enter known design values (pipe lengths, diameters, elevations, demands)
3. Enter field-measured values (pressure readings, flow readings)
4. Receive a system-suggested anomaly location with confidence indication

1.5 Definition of Terms

Term	Definition
Node	A point in the network (Junction, Reservoir, or Tank)
Link	A connection carrying flow between two nodes (Pipe, Pump, or Valve)
Headloss	Pressure/energy lost as water flows through a pipe due to friction
C-factor	Hazen-Williams roughness coefficient; higher = smoother pipe, less friction loss
PRV / PSV / PBV	Pressure Reducing/Sustaining/Breaker Valve
FCV / TCV / GPV	Flow Control / Throttle Control / General Purpose Valve
EPS	Extended Period Simulation — time-varying hydraulic simulation (not used in v1)
GGA	Global Gradient Algorithm — the iterative method used to solve the network equations
Input Parameter	A user-editable design attribute (e.g., pipe length, junction elevation)
Computed Result	A simulation output that is read-only (e.g., pressure, flow rate)
Residual	Difference between measured and simulated expected value at a measurement point

2. Overall Description

2.1 Product Perspective

Audrolics models a water distribution network as a graph: nodes (points with elevation/demand/storage) connected by links (pipes/pumps/valves that carry flow between nodes).

2.2 Tech Stack

Layer	Technology
Frontend	Next.js
Backend/API	FastAPI
Database	MongoDB
Hosting	Vercel (frontend), Render (backend)
Future ML	Modal

2.3 External Interface Requirements

Audrolics is accessed entirely through a modern web browser (latest two versions of Chrome, Edge, or Firefox) at a minimum viewport of 1280×720; no native desktop or mobile application is provided in v1. The frontend (Next.js) communicates with the backend (FastAPI) exclusively over HTTPS REST endpoints; no third-party hydraulic simulation API is used — the solver is implemented in-house per Section 4.1.

2.4 Data Model

Schemas/Collections:

User:

```
{
  "_id": ObjectId,
  "email": "string (unique, verified)",
  "password_hash": "string",
  "created_at": "ISO8601",
  "last_login": "ISO8601",
  "is_verified": "boolean"
}
```

Schematic:

```
{
  "_id": ObjectId,
  "user_id": "ObjectId (ref User)",
  "name": "string",
  "created_at": "ISO8601",
  "updated_at": "ISO8601",
  "threshold_pressure_pct": "number (default 5.0)",
  "threshold_pressure_abs": "number (default 0.5, meters)",
  "threshold_flow_pct": "number (default 10.0)",
  "threshold_flow_abs": "number (default 0.5, L/s)",
  "canvas_state": {
    "zoom": "number",
    "pan": {"x": "number", "y": "number"}
  }
}
```

Node:

```
{
  "_id": ObjectId,
  "schematic_id": "ObjectId (ref Schematic)",
  "label": "string (auto: J-1, R-1, T-1)",
  "type": "enum: JUNCTION | RESERVOIR | TANK",
  "x": "number (canvas px)",
  "y": "number (canvas px)",
  "input_params": {
    "elevation": "number (m)",
    "base_demand": "number (L/s, junction only)",
    "demand_pattern": "string (placeholder for EPS)",
    "diameter": "number (m, tank only)",
    "min_level": "number (m, tank only)",
    "max_level": "number (m, tank only)",
    "initial_level": "number (m, tank only)",
    "total_head": "number (m, reservoir only)"
  },
  "computed": {
    "pressure_head": "number (m, null until simulated)",
  }
}
```

```

        "actual_demand": "number (L/s, null until simulated)",
        "outflow": "number (L/s, reservoir only, null until simulated)",
        "water_level": "number (m, tank only, null until simulated)"
    }
}

Link:
{
    "_id": ObjectId,
    "schematic_id": "ObjectId (ref Schematic)",
    "label": "string (auto: P-1, PU-1, V-1, F-1)",
    "type": "enum: PIPE | PUMP | VALVE | FILTER",
    "from_node_id": "ObjectId (ref Node)",
    "to_node_id": "ObjectId (ref Node)",
    "input_params": {
        "length": "number (m, pipe only)",
        "diameter": "number (mm, pipe/valve only)",
        "roughness": "number (C-factor, pipe only)",
        "minor_loss_coeff": "number (pipe/filter only)",
        "status": "enum: OPEN | CLOSED | ACTIVE",
        "pump_curve": "array of {flow, head} (pump only)",
        "valve_type": "enum (valve only)",
        "valve_setting": "number (valve only, unit depends on type)",
        "gpv_curve": "array of {flow, headloss} (GPV only)",
        "mesh_size": "number (mm, filter only)",
        "filter_status": "enum: CLEAN | PARTIALLY_CLOGGED | CLOGGED"
    },
    "computed": {
        "flow_rate": "number (L/s, null until simulated)",
        "velocity": "number (m/s, pipe only)",
        "headloss": "number (m, null until simulated)",
        "unit_headloss": "number (m/km, pipe only)",
        "head_added": "number (m, pump only)",
        "energy": "number (kWh, pump only)",
        "pressure_drop": "number (m, valve only)"
    }
}

```

Measurement:

```

{
    "_id": ObjectId,
    "schematic_id": "ObjectId (ref Schematic)",
    "element_id": "ObjectId (ref Node or Link)",
    "element_type": "enum: NODE | LINK",
    "measurement_type": "enum: PRESSURE_HEAD | FLOW_RATE",
    "value": "number",
    "unit": "string",
    "timestamp": "ISO8601 (when measured in field)",
    "entered_at": "ISO8601 (when entered into system)"
}

```


3. Feature 1: Schematic Diagram Maker

3.1 Element Library ("weights")

This is the core data model. Each element type has a fixed set of input attributes (weights: what the users enter) and computed attributes (what the simulation produces). This split matters: computed values should never be directly editable, or the anomaly detection in Feature 2 becomes meaningless.

Node Elements

Element	Symbol	Input Parameters	Computed Results
Junction	Small circle	Elevation (m), Base Demand (L/s), Demand Pattern (optional, placeholder for EPS)	Pressure Head (m), Actual Demand (L/s)
Reservoir	Hatched triangle	Total Head / Elevation (m)	Outflow (L/s)
Tank	Rectangle/cylinder	Elevation (m), Diameter (m), Min Level (m), Max Level (m), Initial Level (m)	Hydraulic Head (m), Current Volume (m ³)

Link Elements

Element	Symbol	Input Parameters	Computed Results
Pipe	Straight line	Length (m), Diameter (mm), Roughness Coefficient — Hazen-Williams C-factor (unitless, material-dependent, e.g. ~140 for new PVC, ~100 for old cast iron), Minor Loss Coefficient (≥ 0), Status (Open/Closed)	Flow Rate (L/s), Velocity (m/s), Headloss (m), Unit Headloss (m/km)
Pump	standard pump symbol	Pump Curve (Head vs. Flow) — see note below, or rated Power (kW), Status (On/Off), Speed setting	Flow (L/s), Head Added (m), Energy (kWh)
Valve	valve symbol, typed	Type (PRV – Pressure Reducing, PSV – Pressure Sustaining, PBV – Pressure Breaker, FCV – Flow Control, TCV – Throttle Control, GPV – General Purpose), Diameter (mm), Setting (unit depends on type: pressure [m] for PRV/PSV/PBV, flow [L/s] for FCV, loss coefficient for TCV, curve data), Status (Open/Closed/Active)	Flow (L/s), Headloss/Pressure Drop (m)

Strainer/ Filter	Diamond with dashed line across	Mesh/Screen Size (mm), Minor Loss Coefficient (clean state), Status (Clean/Partially Clogged/Clogged)	Headloss (m)
-----------------------------	--	--	--------------

Curve inputs (Pump Curve, GPV headloss curve): Unlike scalar input parameters, curves require multi-point data (a set of X/Y pairs — e.g., Flow vs. Head for a pump). The right panel must display a dedicated curve-editor sub-widget: a small add/remove-row table with live plotted preview, rather than a plain number field.

Valve Type Conditional UI: each valve type behaves differently in the simulation (PRV caps downstream pressure, FCV caps flow, etc.). The right panel must conditionally display the correct Setting field based on the selected valve type. This is the only part of Feature 1 with significant branching UI logic.

Filter Headloss Model: The filter element uses a state-dependent multiplier on its minor loss coefficient:

- clean: $\text{headloss} = 1.0 \times K_{\text{minor}} \times (v^2/2g)$
- partially_clogged: $\text{headloss} = 3.0 \times K_{\text{minor}} \times (v^2/2g)$
- clogged: $\text{headloss} = 10.0 \times K_{\text{minor}} \times (v^2/2g)$

These multipliers are configurable per-schematic in v1.

3.2 UI/UX

- **Left panel:** element palette (Junction, Reservoir, Tank, Pipe, Pump, Valve). Elements can be dragged or clicked onto the canvas.
- **Right panel (Properties Panel):** appears when an element is selected; displays editable input parameters from Section 3.1. Computed results are displayed in a read-only subsection with distinct visual styling (grayed background, non-editable fields).
- **Top panel:** general styling controls (line color, line thickness, symbol size), canvas zoom controls (50%–200%), fit-to-view button.

- **Parameter-visibility toggle:** a control (checkboxes: Length | Diameter | Pressure | Flow | Elevation) that shows/hides attribute labels directly on the diagram, preventing canvas clutter.
- **Canvas interactions:**
 - snap-to-connect (a pipe's endpoint magnetically locks onto a node when within 10 px),
 - undo/redo(minimum stack depth of 50 actions, persists for current session only),
 - Multi-select: Shift+click or drag-box selection
 - Delete: Delete key or right-click context menu
 - Pan: middle-mouse drag or space+drag
 - Zoom: mouse wheel or +/- buttons
 - Copy/Paste: Ctrl+C / Ctrl+V duplicates selected elements with auto-incremented labels
 - Element auto-labeling: Junctions → J-1, J-2...; Pipes → P-1, P-2...; Pumps → PU-1, PU-2...; Valves → V-1, V-2...; Filters → F-1, F-2...; Reservoirs → R-1, R-2...; Tanks → T-1, T-2...
- **Autosave-lite:**
 - Explicit Save button persists the full diagram (nodes, links, all attributes) to MongoDB
 - Load button reopens saved schematics from the user's personal library
 - Browser localStorage backup: auto-saves current diagram every 30 seconds as a recovery mechanism
 - Browser close warning if unsaved changes exist
- **Export:**
 - PNG snapshot of current diagram view
 - SVG vector export of current diagram view
 - CSV export of simulation results table (all nodes and links)
 - JSON export of full network model (for interoperability)

- **Curve editor:** when a Pump or GPV-type Valve is selected, the right panel swaps the standard numeric Setting field for a small editable table of data point pairs, with a live mini-chart preview.
- **Responsive canvas:** scales to fill available viewport. Right panel width is adjustable via drag handle (min 280 px, max 480 px).

3.3. Input Validation Rules

Attribute	Element(s)	Rule
Elevation	Junction, Reservoir, Tank	Any real number -100 to 5000 m (relative to defined datum)
Base Demand	Junction	≥ 0 L/s
Length	Pipe	> 0 m, $\leq 100,000$ m
Diameter	Pipe, Valve	> 0 mm, typical 12–1200 mm
Roughness (C-factor)	Pipe	1–150; reject 0 or negative; practical minimum 60
Minor Loss Coefficient	Pipe, Filter	≥ 0
Tank Min/Max/Initial Level	Tank	$0 \leq \text{Min} \leq \text{Initial} \leq \text{Max}$
Pump Speed	Pump	> 0 , typically 0.1–2.0
Valve Setting	Valve	> 0 ; unit/range depends on Type; PRV setting must be $<$ upstream pressure and > 0
Filter Mesh Size	Filter	> 0 mm

3.4 Curve Data Validation

For Pump Curves and GPV Headloss Curves:

- Minimum 2 data points required
- Pump curve: head must decrease monotonically as flow increases
- GPV curve: headloss must increase monotonically as flow increases
- No duplicate flow values within a single curve

- Extrapolation behavior: constant beyond the last defined point (do NOT extrapolate). If simulation requires a flow outside the curve range, flag a warning: "Operating point outside pump curve — results may be unreliable."

4. Feature 2: Autocalculate / Anomaly Detection

4.1 Calculation Method

Headloss in pipes computed via the Hazen-Williams equation (industry default):

$$h_f = \frac{K \times L \times Q^{1.852}}{C^{1.852} \times d^{4.87}}$$

Where:

K = constant based on unit system (10.67)

L = pipe length (m),

Q = flow (m³/s),

C = roughness coefficient,

D = diameter (m).

Unit conversion is mandatory. The system shall perform these conversions internally; the user always sees and enters Q in L/s and D in mm.

Worked Example (see Appendix B):

A 100m PVC pipe (C = 140), D = 150mm, carrying Q = 10 L/s:

- $Q_{\text{formula}} = 10 / 1000 = 0.01 \text{ m}^3/\text{s}$

- $d_{\text{formula}} = 150 / 1000 = 0.15 \text{ m}$

- $$h_f = \frac{10.67 \times 100 \times 0.01^{1.852}}{140^{1.852} \times 0.15^{4.87}}$$

- $h_f \approx 0.0674 \text{ m}$

SIMULATION TYPE

Audrolics v1 performs a steady-state simulation — a single snapshot in time, using one fixed set of demands and settings. The Junction element's Demand Pattern attribute (Section 3.1) is captured in the data model as a placeholder for future EPS support but is NOT used in v1 calculations. Every simulation run uses each junction's Base Demand value directly, unmodified by any pattern.

SOLVER METHOD

Because the network can contain loops (not just branched/tree topology), pressures and flows cannot be resolved node-by-node in sequence — they must be solved simultaneously. Audrolics v1 uses the Global Gradient Algorithm (GGA), the same iterative Newton-Raphson method EPANET uses internally.

GGA CONVERGENCE CRITERIA

The solver shall iterate until ALL of the following are satisfied:

- Head imbalance at every node: $|\Delta h| \leq 0.001$ m (absolute tolerance)
- Flow imbalance in every pipe: $|\Delta Q| \leq 0.001$ L/s (absolute tolerance)
- Maximum iterations: $N_{\text{max}} = 200$
- Divergence detection: if the L2 norm of the head correction vector $\|\Delta h\|$ increases for 5 consecutive iterations, abort and report: "Simulation did not converge. Check for unrealistic network configuration (e.g., two reservoirs at different heads connected directly, pump with no downstream resistance)."

If convergence is not achieved within 200 iterations, abort and report the same non-convergence message with a suggestion to review pipe diameters, demands, and connectivity.

The system solves the coupled conservation-of-mass and conservation-of-energy equations across the full network to produce expected pressure/flow at every node and link.

4.2 Workflow

1. Engineer builds or loads a schematic with design input parameters.
2. The system runs the steady-state simulation → expected pressure/flow at every node and link.
3. Actual/measured values are entered per element type:
 - a. Pressure readings (meters of head) are entered at NODE elements (Junction, Tank), matching where pressure gauges are physically installed in the field.
 - b. Flow readings (L/s) are entered at LINK elements (Pipe), matching where flow meters are installed.
 - c. A given measurement point is only valid for the attribute type that element actually produces — the UI shall only offer the applicable input.
4. Engineers enter actual/measured values at multiple points across the network. The more points entered, the more precise the localization. MINIMUM REQUIREMENT: at least 2 measurement points are required for segment narrowing. If fewer than 2 measurements exist, the system flags individual anomalous points but cannot narrow to a segment.
5. System compares expected vs. actual at each measured point, flags all points exceeding the configured deviation threshold (Section 4.3).
6. Segment narrowing algorithm (Section 4.5) processes flagged points to identify the most likely anomalous pipe segment.
7. Results are shown visually on the schematic:
 - a. Measured points are marked with distinct icons
 - b. The suspect segment (pipe(s) between the two nearest bracketing measurement points where the mismatch first appears) is highlighted in red/orange with a tooltip showing confidence score
 - c. A results panel lists all flagged points with residual values

MEASUREMENT CONFLICT HANDLING

If two measurements on the same pipe segment produce contradictory residuals (e.g., one suggests a leak, the other suggests a blockage in the opposite direction), the system shall:

- Flag the segment for MANUAL REVIEW rather than auto-narrowing
- Display both residuals with a warning icon
- Suggest re-checking field measurements

OPTIMAL MEASUREMENT PLACEMENT GUIDANCE

The UI shall provide visual hints (subtle dashed circles) suggesting optimal measurement locations: midpoints of long pipes (>500 m), downstream of valves, and at network extremities.

4.3 Deviation Threshold

The deviation threshold determines how large a gap between expected and measured values must be before a point is flagged as anomalous.

v1 DEFAULTS (configurable per saved schematic, not global):

- Pressure / Head: $\pm 5\%$ OR ± 0.5 m (whichever is LARGER)
- Flow: $\pm 10\%$ OR ± 0.5 L/s (whichever is LARGER)

RATIONALE: Pressure transducers typically have accuracy of $\pm 0.25\text{--}1\%$, while clamp-on ultrasonic flow meters may be $\pm 5\text{--}10\%$. Using a single threshold for both would either miss pressure anomalies or falsely flag normal flow variance. The absolute fallback prevents false positives in low-magnitude systems where a small absolute deviation triggers a large percentage.

The engineer can adjust these thresholds via a settings panel on the diagram. The defaults are starting assumptions for review with a hydraulics SME — real-world sensor accuracy and normal operational variance should inform the final values before field use.

FLAGGING LOGIC

A measurement point is flagged anomalous if:

$$|\text{actual} - \text{expected}| > \max(\text{percentage_threshold} \times |\text{expected}|, \text{absolute_threshold})$$

4.4 Network Validation

Before running a simulation, the system performs a pre-check and blocks the run with a specific error message (naming the offending element) if any of the following are true:

TOPOLOGICAL CHECKS:

- No Reservoir or Tank (water source) exists in the network
- Any node has no connected links (orphaned element)
- Any pipe has an unconnected endpoint (dangling pipe)
- The network contains disconnected subgraphs (isolated clusters not reachable from any source)

HYDRAULIC FEASIBILITY CHECKS (post-simulation or pre-simulation where possible):

- Two reservoirs at different heads connected by an open pipe with no intermediate resistance → infinite flow scenario
- Pump operating with no downstream resistance (dead-head condition)
- All valves in a loop are closed, isolating demand nodes
- Negative pressure (< -10 m) detected at any demand node after simulation (allows minor negative for siphon effects but flags extreme values)
- PRV setting $\geq$ upstream simulated pressure (valve cannot function)
- FCV setting exceeds pipe theoretical capacity

4.5 Anomaly Localization Algorithm

This section formalizes the segment narrowing algorithm described in Section 4.2, Step 6.

DEFINITIONS:

- Let M = set of all measurement points $\{m_1, m_2, \dots, m_n\}$
- For each measurement m_i : $r_i = \text{actual}_i - \text{expected}_i$ (residual)
- A "segment" is the shortest path between two consecutive measurement points along the network graph
- A "bracketing pair" is two measurement points (m_i, m_j) where the residuals indicate a consistent anomaly direction between them

ALGORITHM (pseudocode in Appendix C):

STEP 1 — Compute Residuals

For each measurement point m_i in M :

$$r_i = \text{actual}_i - \text{expected}_i$$

$$\text{flag}_i = |r_i| > \text{threshold}(m_i) \quad [\text{per Section 4.3}]$$

STEP 2 — Identify Bracketing Pairs

For each pair of flagged measurement points (m_i, m_j) connected by a path of one or more pipes: Compute segment residual consistency score:

$$S_{ij} = |r_i - r_j| / L_{ij}$$

where L_{ij} = total pipe length along the path between m_i and m_j

If $\text{sign}(r_i) \approx \text{sign}(r_j)$ AND $|S_{ij}| > \text{segment_threshold}$:

The pair (m_i, m_j) is a candidate bracketing pair.

LEAK SIGNATURE: For a leak between m_i and m_j :

- Upstream pressure residual: $r_{\text{pressure}} < 0$ (lower than expected)
- Downstream pressure residual: $r_{\text{pressure}} < 0$ (lower than expected)
- Flow residual: $r_{\text{flow}} < 0$ (less flow than expected downstream)

BLOCKAGE SIGNATURE: For a partial blockage between m_i and m_j :

- Upstream pressure residual: $r_{\text{pressure}} > 0$ (higher than expected)
- Downstream pressure residual: $r_{\text{pressure}} < 0$ (lower than expected)
- Flow residual: $r_{\text{flow}} < 0$ (reduced flow)

STEP 3 — Rank Candidate Segments

For each candidate bracketing pair, compute confidence score:

$$\text{Confidence} = (|r_i| + |r_j|) / (2 \times \text{threshold}) \times (1 / \text{number_of_pipes_in_segment})$$

Higher confidence = stronger residuals, fewer intermediate pipes

STEP 4 — Handle Multiple Anomalies

If more than 2 non-overlapping candidate segments are flagged:

- Rank all by confidence score
- Highlight the top 3 segments
- Display warning: "Multiple anomaly signatures detected. Review all highlighted segments."

STEP 5 — Node vs. Pipe Anomaly

If a single measurement point is anomalous but no bracketing pair exists:

- Flag the element (node or pipe) containing that measurement
- Display: "Insufficient measurements for segment narrowing. Anomaly may be at [element label] or in an adjacent segment. Add more measurement points for better localization."

CONFIDENCE SCORING

The system presents anomaly flags with a confidence score (0–100%):

- 90–100%: Strong, consistent residuals across multiple bracketing pairs
- 70–89%: Consistent residuals, single bracketing pair
- 50–69%: Weak residuals or borderline threshold
- <50%: Flagged for manual review (conflicting measurements or insufficient data)

5. Feature 3: Landing Page

Static/marketing page:

- Product name and tagline
- Problem statement: manual leak-location calculation is time-consuming and error-prone for field engineers
- Solution overview: diagram → simulate → compare → locate
- Key features list (schematic builder, EPANET-compatible solver, anomaly localization)
- CTA buttons: "Get Started" (Signup) and "Sign In" (Login)
- Footer with team attribution and version number

6. Feature 4: Login & Signup

Authentication exists solely so an engineer can save their schematic diagrams and return to them later — not for access control or team management.

Requirements:

- Email/password signup and login
- Email verification REQUIRED before saving schematics
- Password requirements: minimum 8 characters, at least 1 uppercase letter, 1 lowercase letter, and 1 digit
- Password hashing: bcrypt with salt (work factor ≥ 12)
- Session handling: JWT access token (15-minute expiry) + HTTP-only refresh token cookie (7-day expiry)
- Rate limiting: maximum 5 login attempts per IP address per minute
- Account lockout: temporary 15-minute lock after 10 failed attempts
- Password reset: secure token link sent via email, 24-hour expiry
- Each user has a personal library of saved schematics (list / load / delete)
- No roles, no permission tiers, no admin functions in v1

7. Non-Functional Requirements

Accuracy

1. Simulation results must match EPANET 2.2 output for equivalent inputs within the following tolerances:
 - Node pressure head: ± 0.01 m
 - Link flow rate: ± 0.1 L/s
 - Link headloss: ± 0.01 m
2. Validation must be performed against known EPANET test networks before trusting anomaly detection in the field.

Performance

1. Networks ≤ 50 nodes and ≤ 100 links: simulation must complete in < 2 seconds (95th percentile on a standard laptop with 8 GB RAM)
2. Diagram load from database: < 1 second
3. Save operation: < 500 ms
4. UI response to element selection: < 100 ms

Security

1. Passwords hashed with bcrypt (never stored in plain text)
2. All communications over HTTPS in production
3. JWT access tokens with short expiry (15 minutes)
4. Rate limiting on authentication endpoints (5 attempts/minute)
5. Input sanitization on all user-provided text fields
6. MongoDB connection string and JWT secret stored in environment variables, never in source code

Usability

1. Diagram builder must be usable by engineers without prior CAD experience
2. Tooltips on all input fields explaining the parameter
3. Contextual help panel accessible from any screen
4. Keyboard shortcuts documented in a help modal (Ctrl+?)

Data Integrity

1. Input parameters and computed results must be clearly separated in both the UI (distinct visual sections) and the data model (separate JSON objects)
2. Computed results must be recalculated on every simulation run and must never be persisted as user-editable values
3. Audit log: every simulation run is logged with timestamp, user ID, schematic ID, network size (nodes/links), and result status (success / failure / non-convergence)

Accessibility

1. WCAG 2.1 Level AA compliance
2. All interactive elements keyboard-navigable
3. Color contrast ratio $\geq 4.5:1$ for text
4. Anomaly highlights use pattern/texture in addition to color (for colorblind users)

Browser Compatibility

1. Chrome 120+, Edge 120+, Firefox 121+ (last 2 major versions)

Data Retention

1. Schematics inactive for > 2 years may be archived (configurable)
2. Audit logs retained for 1 year

8. Assumptions & Dependencies

- Hazen-Williams is acceptable as the default headloss method pending confirmation from an actual Maynilad hydraulics SME.
- No live sensor or SCADA integration in v1 — all "actual" values are manually typed by the engineer.
- Network sizes are field-segment scale (≤ 50 nodes), not entire city-wide distribution systems. This assumption affects solver choice and performance targets.
- Water temperature is assumed standard ($\approx 15^{\circ}\text{C}$). Temperature effects on viscosity are negligible for water distribution at v1 scale and may be ignored.
- The system assumes incompressible flow (valid for water distribution networks at typical pressures).

9. Recommendations

The following features were considered but placed out of v1 scope. They represent natural next steps for future versions:

1. Full autosave with conflict resolution
2. Real-time file sharing and multi-user collaboration on a single design
3. ML-based condition monitoring (pattern recognition across historical data, predictive anomaly detection)
4. Hardware integration: automatic field measurement via IoT sensors
5. Auto-import of existing hydraulic designs from AutoCAD or EPANET .inp
6. Support for oil-based hydraulic systems
7. Extended Period Simulation (EPS) with time-varying demands
8. Uncertainty quantification: probabilistic confidence intervals for anomaly flags rather than binary thresholds

10. Functional Requirements

FR-1: The system shall allow users to place and connect Junction, Reservoir, Tank, Pipe, Pump, Valve, and Strainer/Filter elements on a canvas with snap-to-connect behavior.

FR-2: The system shall allow editing of each element's input parameters via a right-side properties panel, including a dedicated curve-editor widget for Pump and GPV-type Valve elements.

FR-3: The system shall validate all input parameters against the rules in Section 3.3 and Section 3.4, and block Save/Run actions on invalid data with inline error messages.

FR-4: The system shall run a steady-state hydraulic simulation using the Global Gradient Algorithm upon user request, after passing the network validation checks in Section 4.4.

FR-5: The system shall accept manually entered measured values at node elements (pressure head in meters) and link elements (flow rate in L/s), per Section 4.2.

FR-6: The system shall flag measured points that deviate from simulated expected values beyond the configured threshold (Section 4.3), with separate thresholds for pressure and flow.

FR-7: The system shall narrow flagged deviations to a specific candidate pipe segment when multiple measurement points are available, using the algorithm in Section 4.5, and highlight that segment on the schematic with a confidence score.

FR-8: The system shall allow authenticated users to save, load, and delete their own schematic diagrams (Section 6).

FR-9: The system shall export the current diagram as PNG, SVG, CSV (results table), or JSON (full network model) on request.

FR-10: The system shall present a landing page describing the product and problem it solves, with CTA buttons to Login and Signup.

FR-11: The system shall provide undo/redo functionality with a minimum stack depth of 50 actions.

FR-12: The system shall auto-label elements (J-1, P-1, etc.) and allow users to rename them.

FR-13: The system shall warn users before closing the browser tab if unsaved changes exist.

FR-14: The system shall store a localStorage backup of the current diagram every 30 seconds.

11. API Specification

Base URL: <https://api.audrolics.com/api/v1>

Authentication

All endpoints except /auth/* require Bearer token in Authorization header.

POST /auth/signup

Body: { "email": "string", "password": "string" }

Response: { "user_id": "string", "message": "Verification email sent" }

Errors: 400 (invalid email/format), 409 (email exists), 429 (rate limited)

POST /auth/verify

Body: { "token": "string" }

Response: { "verified": true }

POST /auth/login

Body: { "email": "string", "password": "string" }

Response: { "access_token": "string", "refresh_token": "string" }

Errors: 401 (invalid credentials), 429 (rate limited), 403 (account locked)

POST /auth/refresh

Body: { "refresh_token": "string" }

Response: { "access_token": "string" }

POST /auth/reset-password-request

Body: { "email": "string" }

Response: { "message": "Reset link sent if email exists" }

POST /auth/reset-password

Body: { "token": "string", "new_password": "string" }

Schematics

GET /schematics

Response: [{ "id": "string", "name": "string", "updated_at": "ISO8601" }]

POST /schematics

Body: { "name": "string", "nodes": [...], "links": [...], "thresholds": {...} }

Response: { "id": "string", "created_at": "ISO8601" }

GET /schematics/{id}

Response: Full schematic document (nodes, links, measurements, thresholds)

PUT /schematics/{id}

Body: Same as POST

Response: { "updated_at": "ISO8601" }

DELETE /schematics/{id}

Response: 204 No Content

Simulation

POST /simulate

Body: { "nodes": [...], "links": [...] }

Response: {

```
"status": "success | non_convergence | validation_error",
"node_results": [{ "id": "string", "pressure_head": "number", ... }],
"link_results": [{ "id": "string", "flow_rate": "number", ... }],
"iterations": "number",
"max_head_error": "number",
"max_flow_error": "number"
```

}

Errors: 400 (validation failure with specific error code), 422 (non-convergence)

Anomaly Detection

POST /anomalies

Body: {

```
"schematic_id": "string",
"measurements": [{ "element_id": "string", "type":
"PRESSURE_HEAD|FLOW_RATE", "value": "number" }]
```

}

```

Response: {
  "flagged_points": [{ "element_id": "string", "expected": "number", "actual":
    "number", "residual": "number" }],
  "suspect_segments": [{ "from": "string", "to": "string", "confidence": "number
    (0-1)", "signature": "LEAK|BLOCKAGE|UNKNOWN" }],
  "warnings": ["string"]
}

```

Export

GET /export/{schematic_id}?format=png|svg|csv|json

Response: Binary file (PNG/SVG) or text (CSV/JSON)

12. Error Handling

Standard error response format:

```

{
  "error_code": "E###",
  "message": "Human-readable description",
  "element_id": "string (if applicable)",
  "attribute": "string (if applicable)"
}

```

Error Codes:

E100 — No water source

"Network has no Reservoir or Tank. Add at least one source node."

E101 — Orphaned node

"Node '{label}' has no connected links. Connect it or remove it."

E102 — Dangling pipe

"Pipe '{label}' has an unconnected endpoint. Connect both ends to nodes."

E103 — Disconnected subgraph

"Network contains isolated elements not reachable from any source."

E104 — Simulation non-convergence

"Simulation did not converge after {N} iterations. Check for unrealistic configurations (e.g., reservoirs at different heads connected directly)."

E105 — Negative pressure detected

"Negative pressure ({value} m) detected at node '{label}'. Check elevations and demands."

E106 — Pump curve out of range

"Pump '{label}' operating point ({flow} L/s, {head} m) is outside its curve. Results may be unreliable."

E200 — Invalid input value

"Invalid value for {attribute} on {element} '{label}': {value}. Expected: {rule}."

E201 — Curve validation failed

"{element} '{label}' curve: {reason} (e.g., 'head must decrease monotonically')."

E202 — Tank level violation

"Tank '{label}': Min Level ({min}) must be $\leq$ Initial Level ({initial}) $\leq$ Max Level ({max})."

E203 — PRV setting invalid

"PRV '{label}' setting ({setting} m) must be less than upstream pressure ({upstream} m) and greater than 0."

E300 — Insufficient measurements

"At least 2 measurement points are required for segment narrowing. {N} point(s) provided."

E301 — Measurement conflict

"Conflicting residuals detected on segment {from}—{to}. Manual review required."

E400 — Authentication required

"Please log in to perform this action."

E401 — Invalid credentials

"Email or password is incorrect."

E403 — Account locked

"Account temporarily locked due to multiple failed attempts. Try again in {N} minutes."

E404 — Schematic not found

"Schematic '{id}' does not exist or you do not have permission to access it."

E429 — Rate limited

"Too many requests. Please wait {N} seconds before retrying."

13. Testing Requirements

13.1 Simulation Accuracy Tests

Before field deployment, the solver must pass the following validation networks against EPANET 2.2:

TEST-NET-01: Simple Branch

- 1 Reservoir, 3 Junctions, 2 Pipes
- Purpose: Sanity check for basic headloss calculation
- Tolerance: head ± 0.01 m, flow ± 0.1 L/s

TEST-NET-02: Single Loop

- 1 Reservoir, 4 Junctions, 4 Pipes (forming a square loop)
- Purpose: Verify loop flow distribution using GGA
- Tolerance: head ± 0.01 m, flow ± 0.1 L/s

TEST-NET-03: EPANET Example 1 (Net1)

- Standard EPANET validation network
- Purpose: Full solver validation against industry standard
- Tolerance: head ± 0.01 m, flow ± 0.1 L/s

TEST-NET-04: Pump Network

- 1 Reservoir, 1 Pump, 2 Junctions, 1 Pipe
- Purpose: Verify pump curve interpolation
- Tolerance: head ± 0.01 m, flow ± 0.1 L/s, pump head ± 0.01 m

TEST-NET-05: Valve Network

- 1 Reservoir, 1 PRV, 2 Junctions, 2 Pipes

- Purpose: Verify PRV pressure capping behavior
- Tolerance: downstream pressure ± 0.01 m

13.2 Anomaly Detection Tests

TEST-ANO-01: Known Leak Localization

- Use TEST-NET-01 with a known "leak" simulated by increasing roughness on Pipe P-2
- Enter measurements at all nodes
- Expected: System highlights Pipe P-2 with confidence > 70%

TEST-ANO-02: Blockage Localization

- Use TEST-NET-02 with a partial blockage simulated on one pipe
- Expected: System identifies correct segment with blockage signature

TEST-ANO-03: Insufficient Measurements

- Use TEST-NET-01 with only 1 measurement point
- Expected: System flags the point but displays E300 for segment narrowing

TEST-ANO-04: Conflicting Measurements

- Inject contradictory residuals on the same segment
- Expected: System displays E301 and highlights for manual review

13.3 UI/UX Tests

- Undo/redo stack: perform 60 actions, verify undo works for all 60
- Copy/paste: duplicate a complex sub-network, verify labels auto-increment
- Export: verify PNG, SVG, CSV, and JSON exports produce valid files
- Responsive: verify canvas scales correctly at 1280×720, 1920×1080, and 2560×1440

13.4 Security Tests

- Password complexity: reject passwords failing rules

- Rate limiting: verify 429 response after 5 failed login attempts
- JWT expiry: verify access token expires after 15 minutes
- SQL/NoSQL injection: verify all user inputs are sanitized

14. References

- [1] Rossman, L., Woo, H., Tryby, M., Shang, F. EPANET 2.2 User Manual, U.S. Environmental Protection Agency, 2020.
- Section 3.1: Hydraulic Simulation Model (GGA formulation)
 - Section 3.2: Headloss Formulas (Hazen-Williams)
 - Appendix D: Units and Physical Properties
 - Appendix E: Input File Format (.inp)
- [2] Romero Ben, L., Alves, D., Blesa Izquierdo, J., Cembrano Gennari, G., Puig Cayuela, V., & Duviella, E. (2023). Leak detection and localization in water distribution networks: Review and perspective. *Annual Reviews in Control*, 55, 392–419.
- [3] Todini, E., & Pilati, S. (1988). A gradient algorithm for the analysis of pipe networks. In *Computer Applications in Water Supply* (Vol. 1, pp. 1–20). John Wiley & Sons.
- Original GGA formulation
- [4] ISO/ASCE reference tables for Hazen-Williams C-factor by pipe material.
- [5] Additional leak-detection and anomaly-localization literature:
- Graph-based localization methods ML-based condition monitoring surveys
 - Pressure transient analysis for leak detection

Appendix A: Units & Convention

Attributes intentionally mix meters and millimeters (elevation/length vs. diameter), matching common field practice. Implementers must NOT silently normalize all values to one unit, as this would break the formulas in Section 4.1. Internal conversion is performed at calculation time.

Quantity	Unit
Length, Elevation	meters (m)
Diameter	millimeters (mm)
Flow	liters/second (L/s)
Pressure / Head	meters of head (m) bar shown via UI toggle (1 bar $\approx$ 10.2 m)
Power	kilowatts (kW)
Energy	kilowatt-hours (kWh)
Velocity	meters/second (m/s)
Volume	cubic meters (m ³)

Appendix B: Hazen-Williams Worked Example

EXAMPLE 1: Single PVC Pipe

Given:

Pipe length (L) = 100 m

Diameter (D) = 150 mm

Roughness (C) = 140 (new PVC)

Flow rate (Q_{input}) = 10 L/s

Step 1 — Convert units for formula:

$Q_{\text{formula}} = 10 / 1000 = 0.01 \text{ m}^3/\text{s}$

$d_{\text{formula}} = 150 / 1000 = 0.15 \text{ m}$

Step 2 — Compute headloss:

$$h_f = (10.67 \times 100 \times 0.01^{1.852}) / (140^{1.852} \times 0.15^{4.87})$$

$$\text{Numerator: } 10.67 \times 100 \times 0.01^{1.852} = 1067 \times 0.0001412 = 0.1507$$

$$\text{Denominator: } 140^{1.852} \times 0.15^{4.87} = 6317.6 \times 0.0003538 = 2.235$$

$$h_f = 0.1507 / 2.235 = 0.0674 \text{ m}$$

Step 3 — Compute velocity:

$$A = \pi \times (0.15/2)^2 = 0.01767 \text{ m}^2$$

$$v = Q/A = 0.01 / 0.01767 = 0.566 \text{ m/s}$$

Step 4 — Unit headloss:

$$h_f \text{ per km} = 0.0674 \times (1000/100) = 0.674 \text{ m/km}$$

Appendix C: Anomaly Localization Pseudocode

```
function detectAnomalies(schematic, measurements, thresholds):
    // Step 1: Compute residuals
    flagged = []
    for m in measurements:
        element = findElement(schematic, m.element_id)
        expected = element.computed[m.type]
        actual = m.value
        residual = actual - expected

        threshold = getThreshold(m.type, thresholds)
        if abs(residual) > threshold:
            flagged.append({
                element: element,
                expected: expected,
                actual: actual,
                residual: residual,
                type: m.type
            })
```

```

    // Step 2: If fewer than 2 flagged points, return point-level flags
    only
    if flagged.length < 2:
        return {
            flagged_points: flagged,
            suspect_segments: [],
            warnings: ["E300: Insufficient measurements for segment
narrowing"]
        }

    // Step 3: Find candidate bracketing pairs
    candidates = []
    for i in range(0, flagged.length):
        for j in range(i+1, flagged.length):
            path = shortestPath(flagged[i].element, flagged[j].element)
            if path is None:
                continue

            segment_length = sum(pipe.length for pipe in path.pipes)
            consistency = abs(flagged[i].residual - flagged[j].residual) /
segment_length

            // Check for consistent anomaly signature
            signature = classifySignature(flagged[i], flagged[j], path)
            if signature != "INCONSISTENT" and consistency >
SEGMENT_THRESHOLD:
                confidence = calculateConfidence(
                    flagged[i], flagged[j], path, thresholds
                )
                candidates.append({
                    from: flagged[i].element,
                    to: flagged[j].element,
                    path: path,
                    signature: signature,
                    confidence: confidence,
                    consistency: consistency
                })

    // Step 4: Handle conflicts
    warnings = []

```

```

if hasOverlappingConflicts(candidates):
    warnings.append("E301: Conflicting measurements detected")
    candidates = filterNonConflicting(candidates)

// Step 5: Rank and return
candidates.sort(by: confidence, descending)
return {
    flagged_points: flagged,
    suspect_segments: candidates.slice(0, 3),
    warnings: warnings
}

function classifySignature(m1, m2, path):
    // Leak: both residuals negative (lower than expected)
    // Blockage: upstream positive, downstream negative
    if m1.residual < 0 and m2.residual < 0:
        return "LEAK"
    elif m1.residual > 0 and m2.residual < 0:
        return "BLOCKAGE"
    elif m1.residual < 0 and m2.residual > 0:
        return "BLOCKAGE_REVERSE"
    else:
        return "INCONSISTENT"

function calculateConfidence(m1, m2, path, thresholds):
    avg_residual = (abs(m1.residual) + abs(m2.residual)) / 2
    threshold_avg = (thresholds[m1.type] + thresholds[m2.type]) / 2
    residual_score = min(avg_residual / threshold_avg, 2.0) / 2.0
    pipe_penalty = 1.0 / max(path.pipes.length, 1)
    return min((residual_score * 0.7 + pipe_penalty * 0.3) * 100, 100)

```